Threads

Chapter 4

Organization

- An Introduction
- Multithreading Models
- Thread Libraries
- Threading Issues

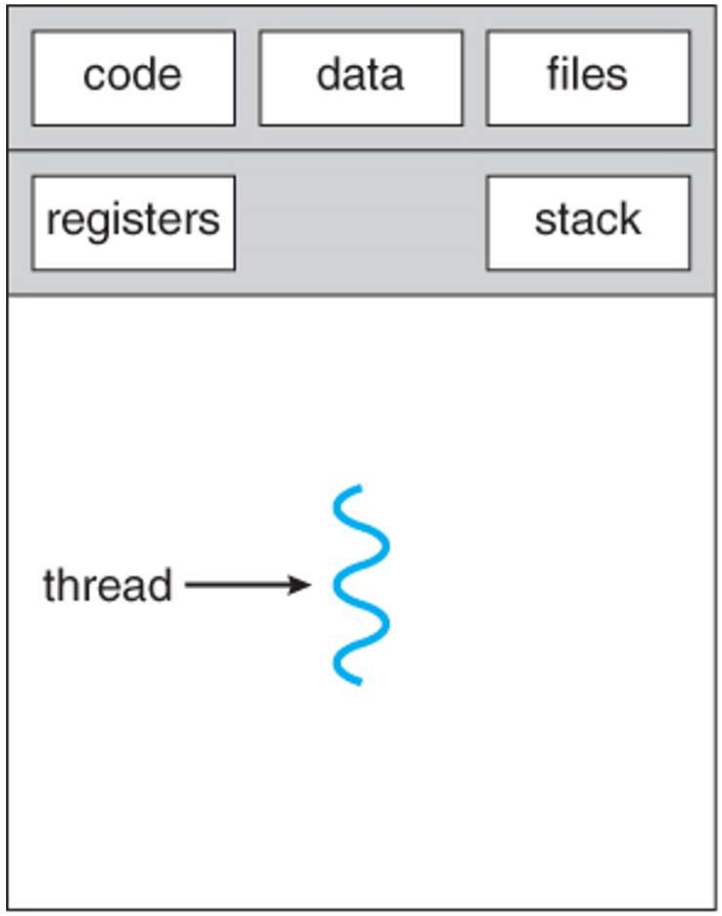


Organization

- **An Introduction**
- Multithreading Models
- Thread Libraries
- Threading Issues



Two Characteristics of Processes



single-threaded process

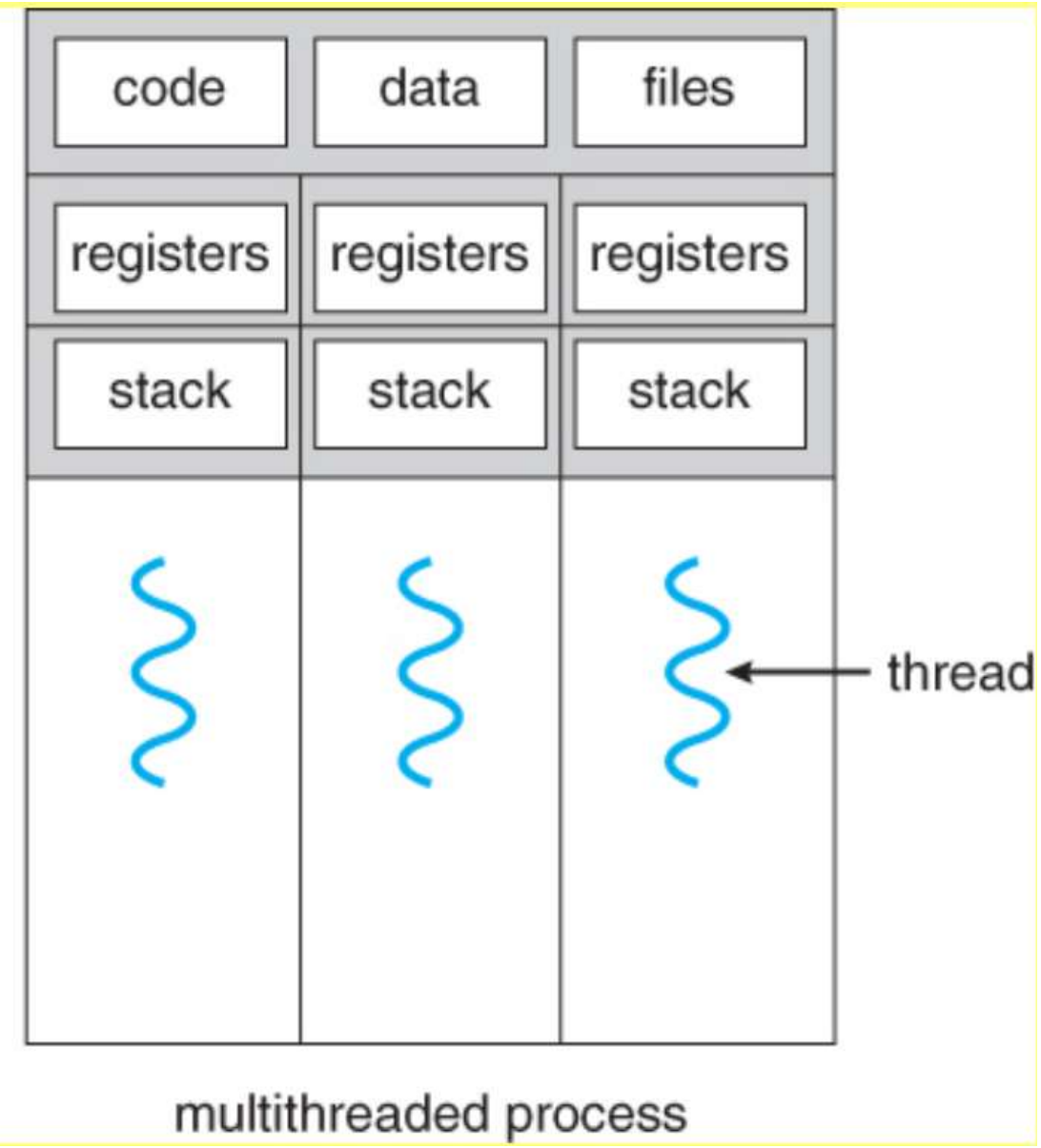
Resource ownership: process is allocated a virtual address space to hold the process image

Scheduling/execution: follows an execution path that may be interleaved with other processes

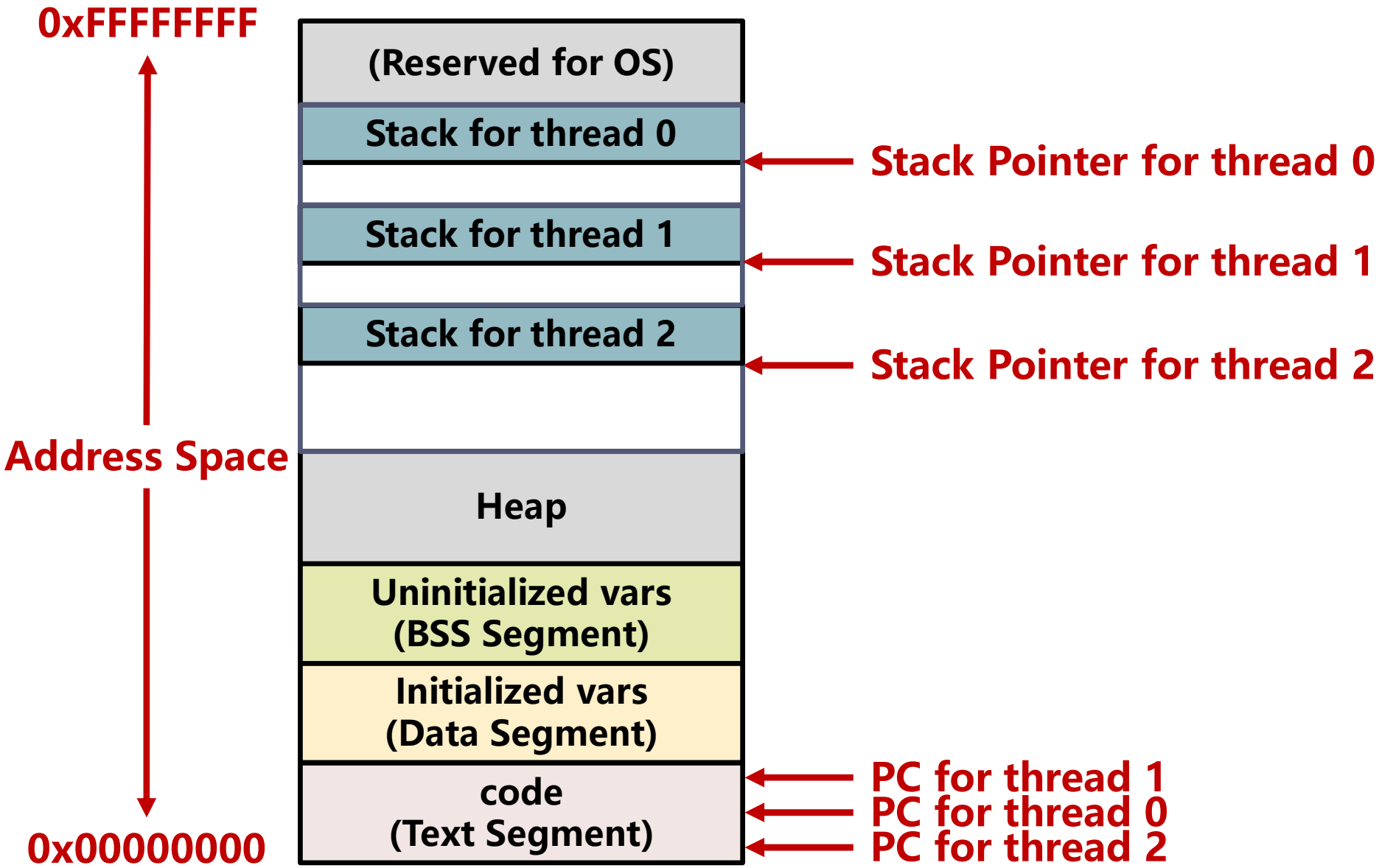


Threads and Processes

- The unit of dispatching is referred to as a **thread** (or **lightweight process**)
- The unit of resource ownership is referred to as a **process** or **task**



Address Space with Multiple Threads



Thread

- A thread is a basic unit of CPU utilization; It comprises of :
  - A **thread ID**
  - A **program counter**
  - A **register set**
  - A **stack**
- A thread shares with other threads belonging to the same process:
  - **Code section**
  - **Data section and Heap**
  - Other OS resources such as **opened files** and **signals**



Process Items vs. Thread Items

Per process items	Per thread items
Address Space Global Variables Open files Child Processes Pending alarms Signals and signal handlers Accounting information	Thread Context (Registers, Program Counter) Stack Execution State



Each Thread Has Its Own Stack

- Each thread’ s stack contains one frame for each procedure called but not yet returned from.
  - This frame contains the **procedure’ s local variables** and the **return address** to use when the procedure call has finished.
  - **Each thread** will generally call different procedures and thus. **have a different execution history**



Context Switching

- **Thread switch** is a type of context switching from one thread to another thread in the same process.
  - It is faster and has lower overhead, compared with process switch
  - Reason: Thread context switches do not need updating memory management structures
- **Process switch** is also type of context switching from one process to another process.
  - It involves updating memory management structures to switch between separate address spaces
  - Thus provide better isolation between processes.

Thread switching occurs within the same process, leveraging shared virtual address spaces to minimize hardware overhead — **no page table updates**, **no TLB flushes**, **maximum cache locality**.



TCS versus PCS

	Thread Context Switch	Process Context Switch
1	TCS occurs when CPU saves the current state of one thread and switches to another thread of the same process	PCS occurs when OS scheduler saves the current state of the running process (including the state of PCB) and switches to another process
2	TCS does **not** involves switching of memory address spaces. All the memory addresses that the processor accounts remain saved	PCS involves **switching of memory address spaces**. All the memory addresses that the processor accounts gets flushed
3	Processor’ s **cache and TLB preserve** their state	Processor’ s **cache and TLB** gets **flushed**.
4	**TCS** is a bit **faster and cheaper** because it involves only switching of registers and stack pointers.	**PCS** is relatively **slower and costlier**, because it involves the heavy cost of changing the address space.

Overhead Comparison

Switch Type	Address Space Change	TLB Flush Required	Relative Overhead	Primary Trigger
Mode Switch	No	No	Very Low	System calls, Interrupts
Thread Switch	No	No	Low	Time slice, I/O wait, Yield
Process Switch	**Yes**	**Yes**	**High**	Process termination, IPC, Fork

Mode and thread switches benefit from shared address spaces, **Process switches** incur severe penalties due to memory isolation and TLB invalidation

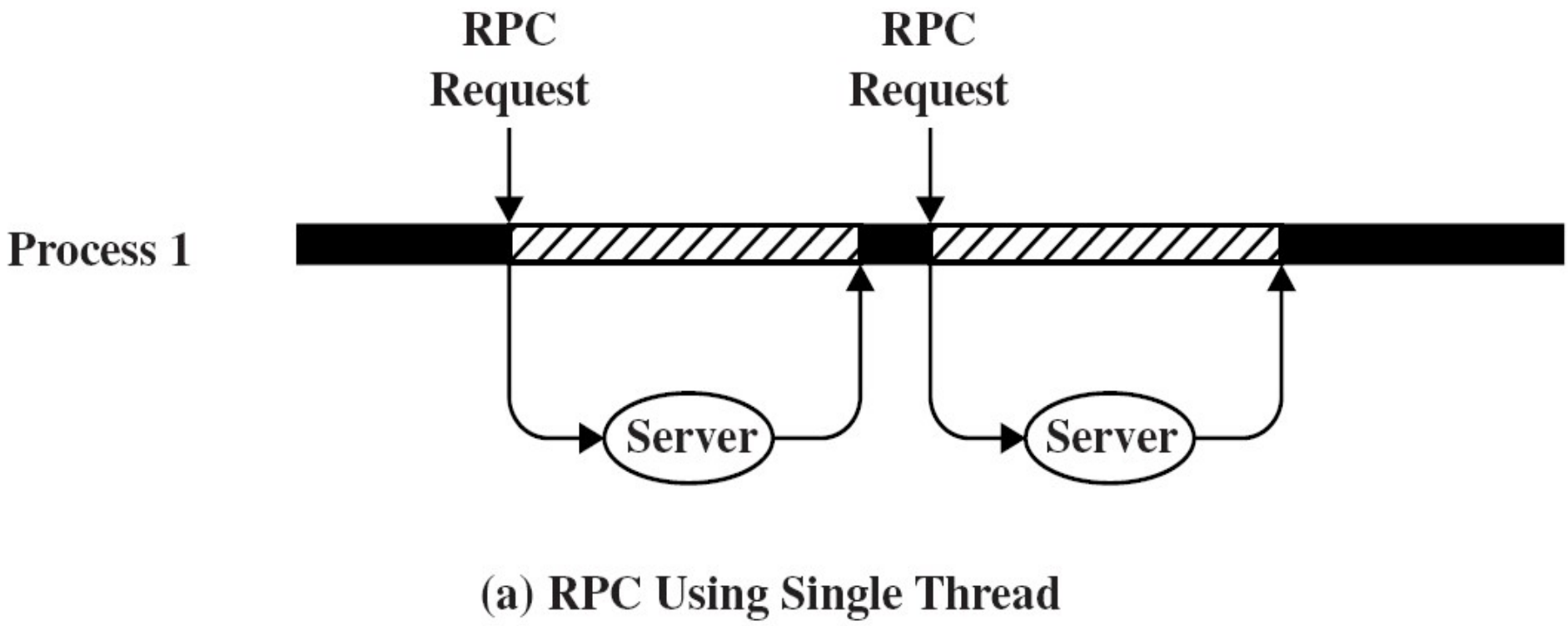


Example (1) Scenario

- A program performs two remote procedure calls (远程过程调用) to two different hosts to obtain a combined results
- Single-Threaded implementation
  - The results are obtained in sequence→the program has to wait for a response from each host in turn
- Multi-Threaded implementation
  - Uses a separate thread for each RPC→results in a substantial speedup.



Example (2)
Single-Threaded Implementation



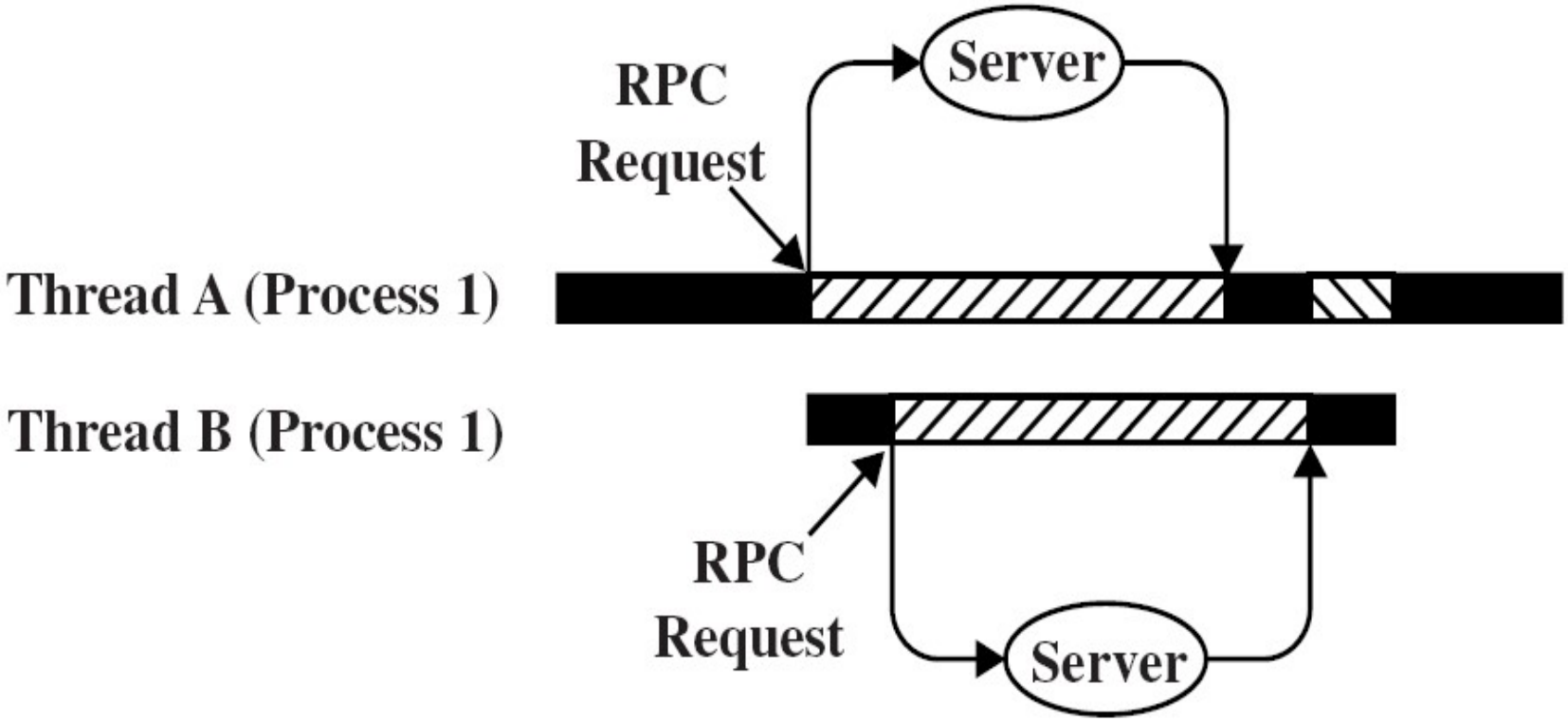
(a) RPC Using Single Thread

Blocked, waiting for response to RPC

Blocked, waiting for processor, which is in use by Thread B

Running

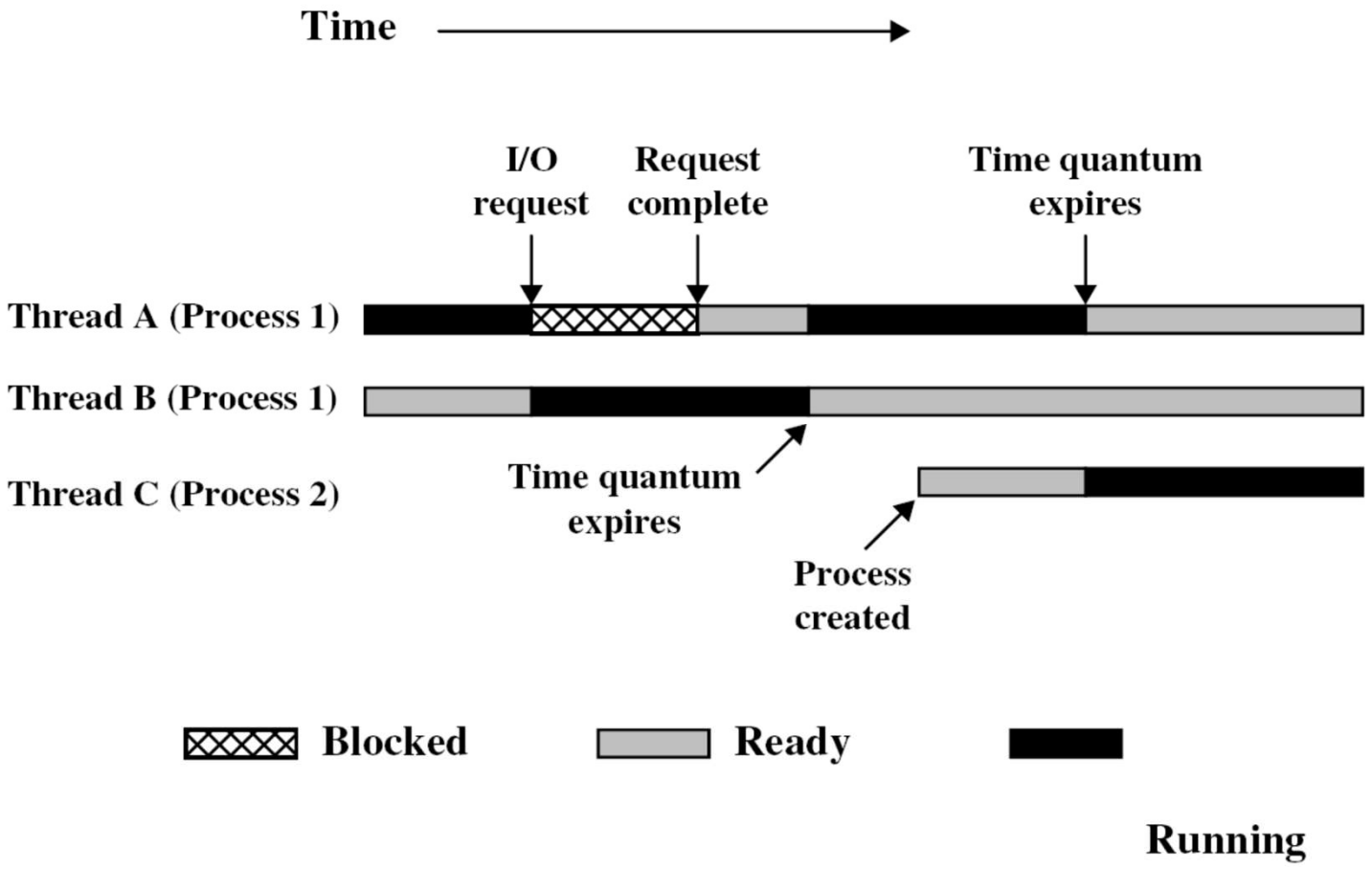
Example (3)
Multithreaded Implementation



(b) RPC Using One Thread per Server (on a uniprocessor)

Interleaving Execution

- One a uniprocessor, multiprogramming enables the interleaving of multiple threads within multiple processes



Why Do We Need Threads?
Benefits of Multi-threaded Programming

- **Responsiveness:**
  - **In interactive application, user interaction in a thread can be executing, even when another thread is blocked or is performing a lengthy operation**
- **Resource Sharing:**
  - **The memory and the resources of a process are shared among its threads.**
- **Economy:**
  - **It is more economical to create and context-switch threads, and communicate between threads of the same process.**
- **Scalability-Utilization of Multiprocessor Architectures:**
  - **Multiple threads may be running in parallel on different processors.**



Benefits of Threads (Compared to Processes)

Benefits, compared with multiple processes

- Less time to create a new thread than a process
- Less time to terminate a thread than a process
- Less time to switch between two threads within the same process
- Since threads within the same process share memory and files, they can communicate with each other without invoking the kernel
- May **speed up** the application!!
  - (Multiple CPUs) or (overlap I/O and computing)

Benefits same as multiple processes

Multithreading (1)
多线程

- What is Multithreading
  - The ability to support multiple threads of execution within a single process
  - In a multithreaded environment, a process is defined as the unit of resource allocation and a unit of protection.
- Protection between threads within a single process?
  - Not necessary!! Why?



Multithreading (2)

- Classification
  - How many address spaces does the system have?
    - One or Many?
  - How many threads in each address space?
    - One or Many?



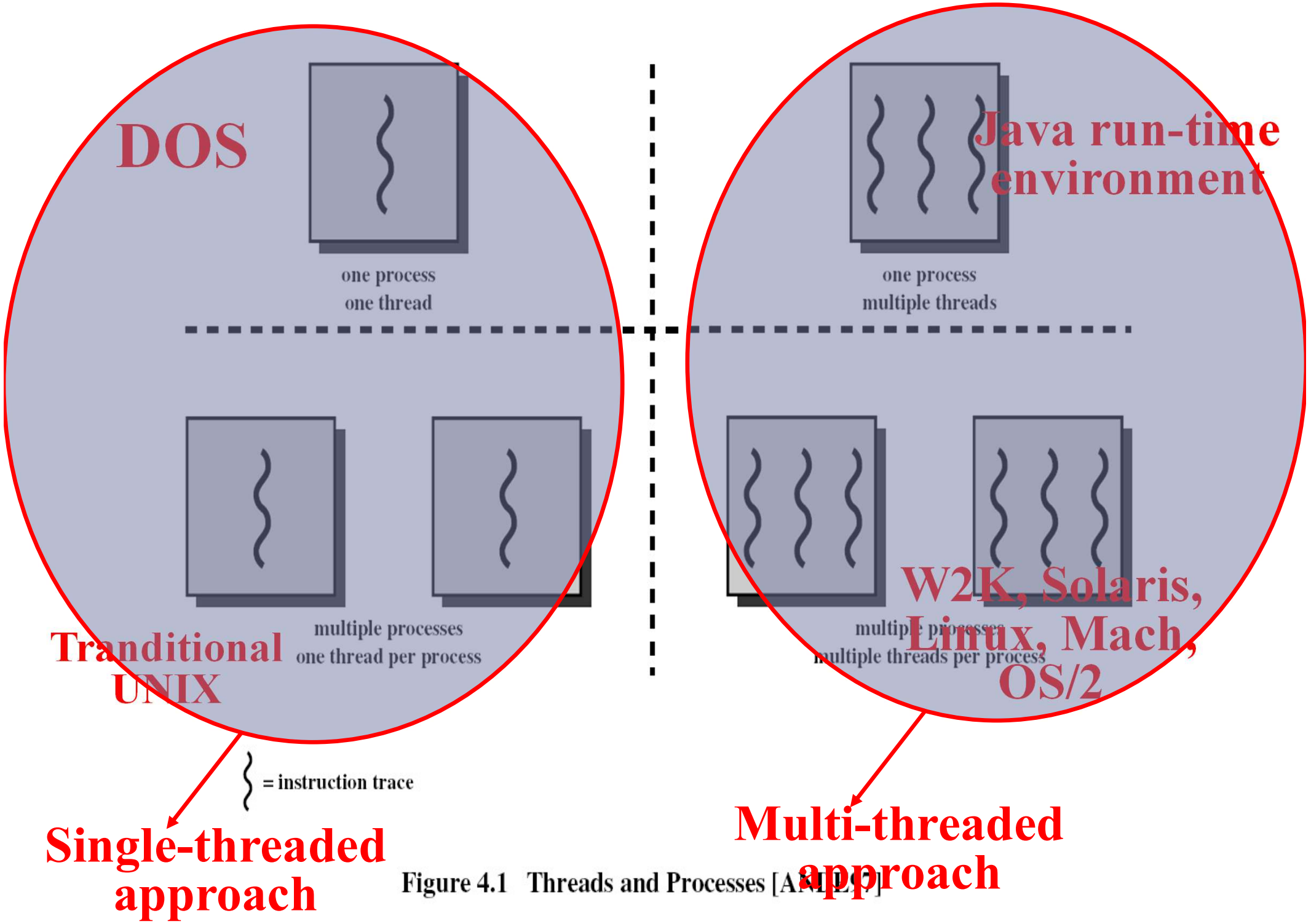
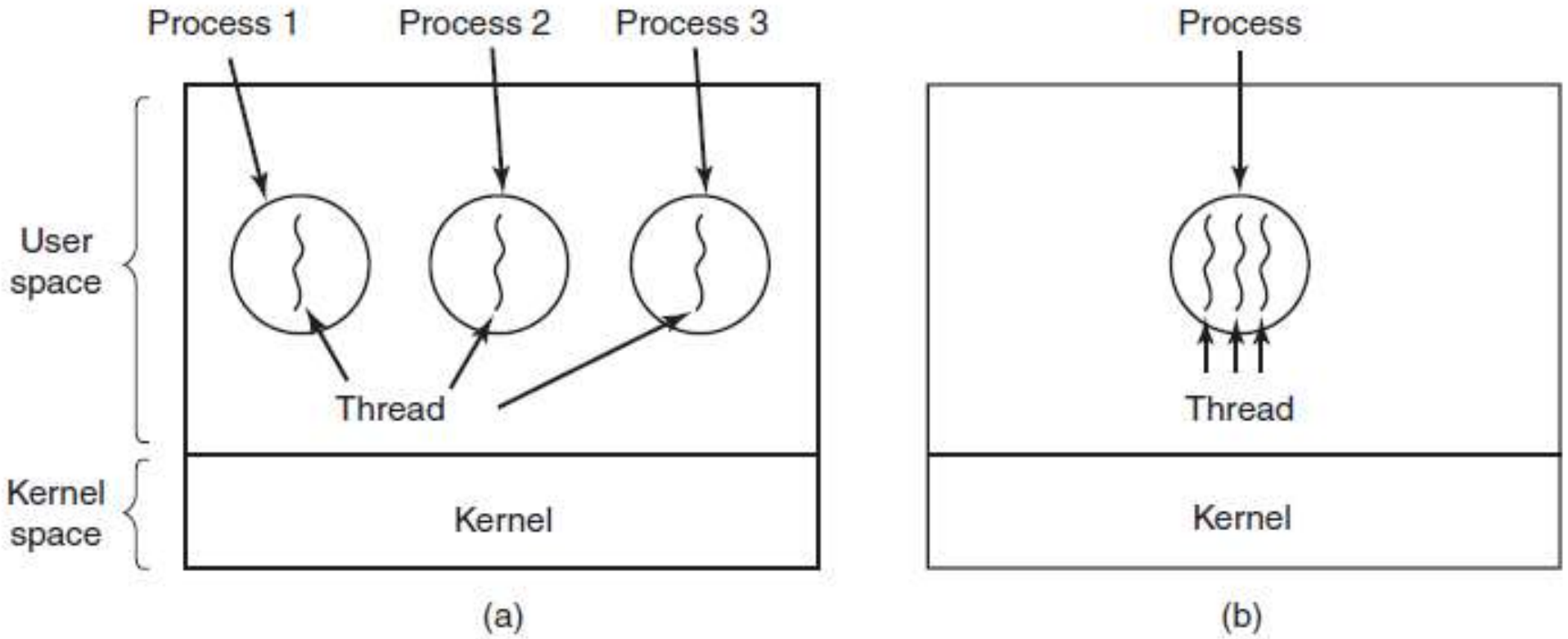


Figure 4.1 Threads and Processes [AN


Process 1
Process 2
Process 3
User space
Thread
Kernel space
Kernel
(a)
Process
Thread
Kernel
(b)

单线程进程模型

多线程进程模型

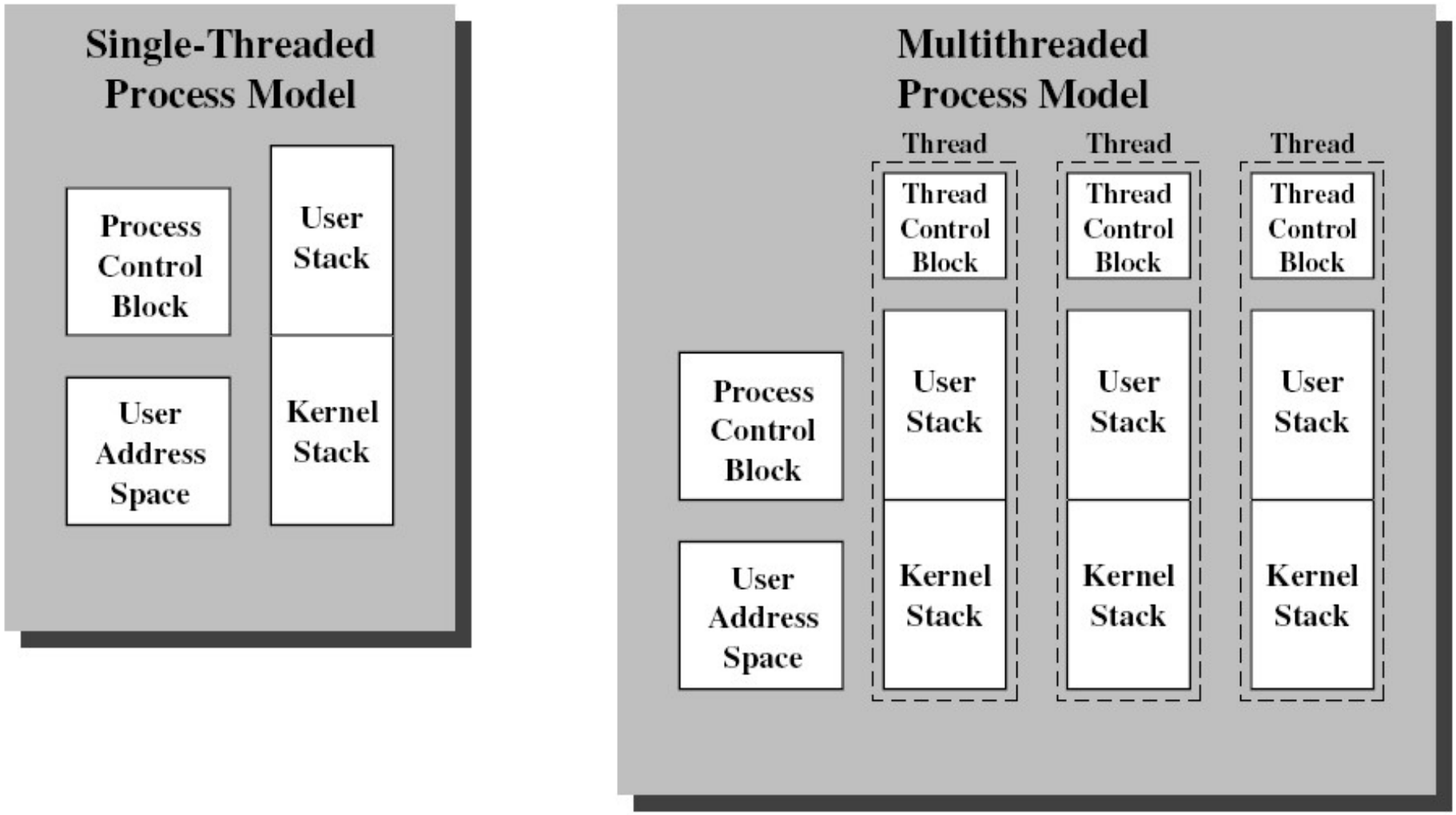


Figure 4.2 Single Threaded and Multithreaded Process Models

Scheduling and Dispatching

- Scheduling and dispatching is done on thread basis, in the OS that supports threads
  - Most of the state information dealing with execution is maintained in thread-level data structures
- Some actions that affect all of the threads in a process:
  - Suspension (swapping the address space out of main memory)
  - Termination of a process (terminates all threads within the process)

These actions must be managed by OS in process-level



Thread Functionality (1)
Thread States

- **Thread States:** Running, Ready, Blocked
- Operations associated with a change in thread state
  - **Spawn (派生):** Spawn another thread
  - **Block (阻塞):** when a thread needs to wait for an event, it will block.
  - **Unblock (解除阻塞):** when a event for which a thread is blocked occurs, the thread is moved to the ready queue
  - **Finish (结束):** Deallocate register context and stacks



Thread Functionality (2)
Thread Synchronization

- **All of the threads of a process share the same address space and other resources**
  - **Any alteration of a resource by one thread affects the environment of the other threads in the same process**
  - **Therefore, it is necessary to synchronize the activities of the various threads so that they do not interfere with each other.**
- **Details will be discussed in the next two chapters.**



Organization

- An Introduction
- **Multithreading Models**
- Thread Libraries
- Threading Issues



User-Level and Kernel-Level Threads

- Two broad categories of thread implementation:
  - **User-level threads** (ULTs)
  - **Kernel-level threads** (KLTs)
    - Also called as kernel-supported threads or lightweight processes.

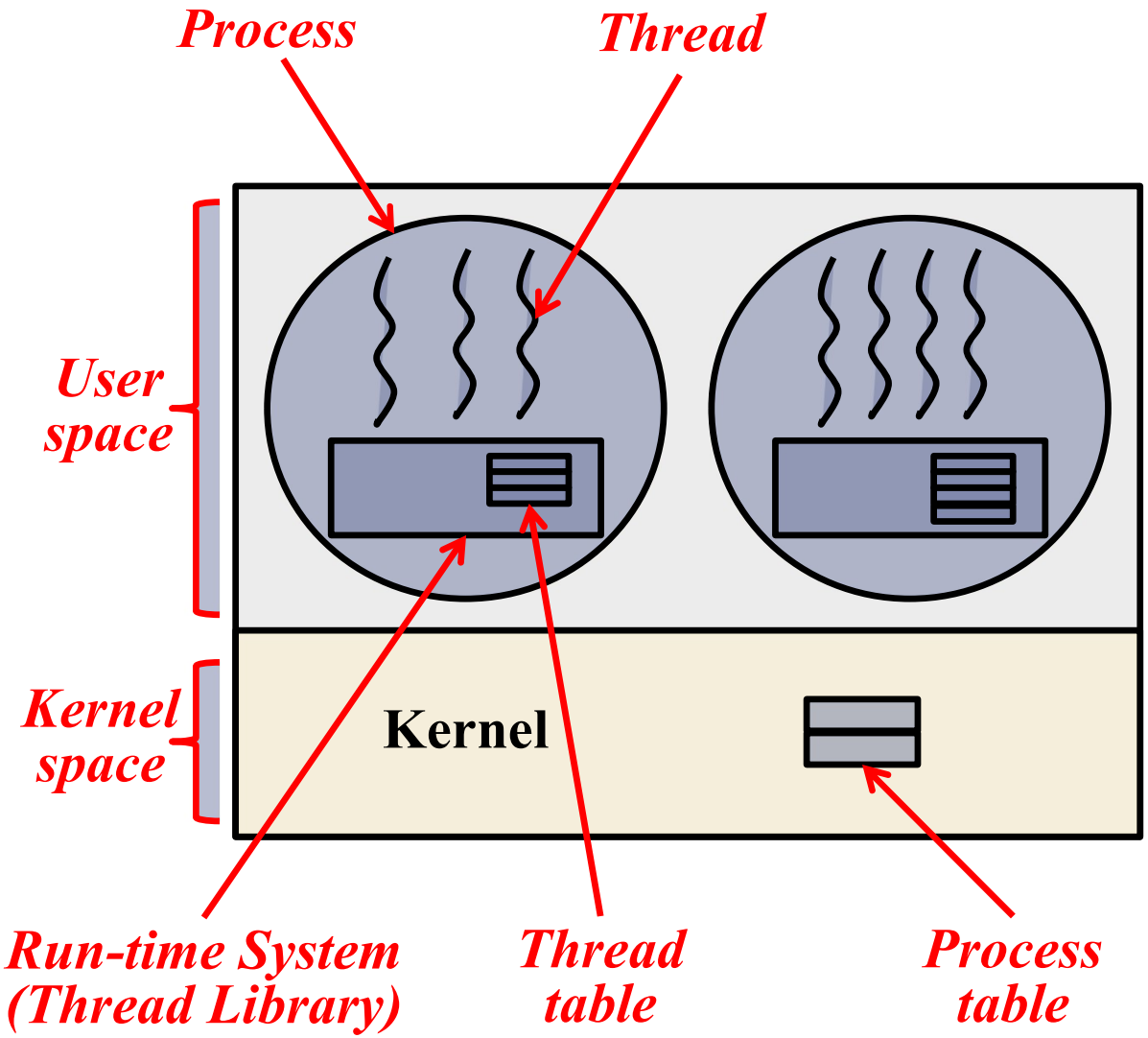
A hybrid implementation is also available



User-Level Threads

- **Pure ULT: All thread management is done by the application**
  - **The kernel is not aware of the existence of threads**
- **Threads library: a package of routines for ULT management**
  - **The threads library contains code for creating and destroying threads**
  - **For passing messages and data between threads**
  - **For scheduling thread execution**
  - **For saving and restoring thread context**





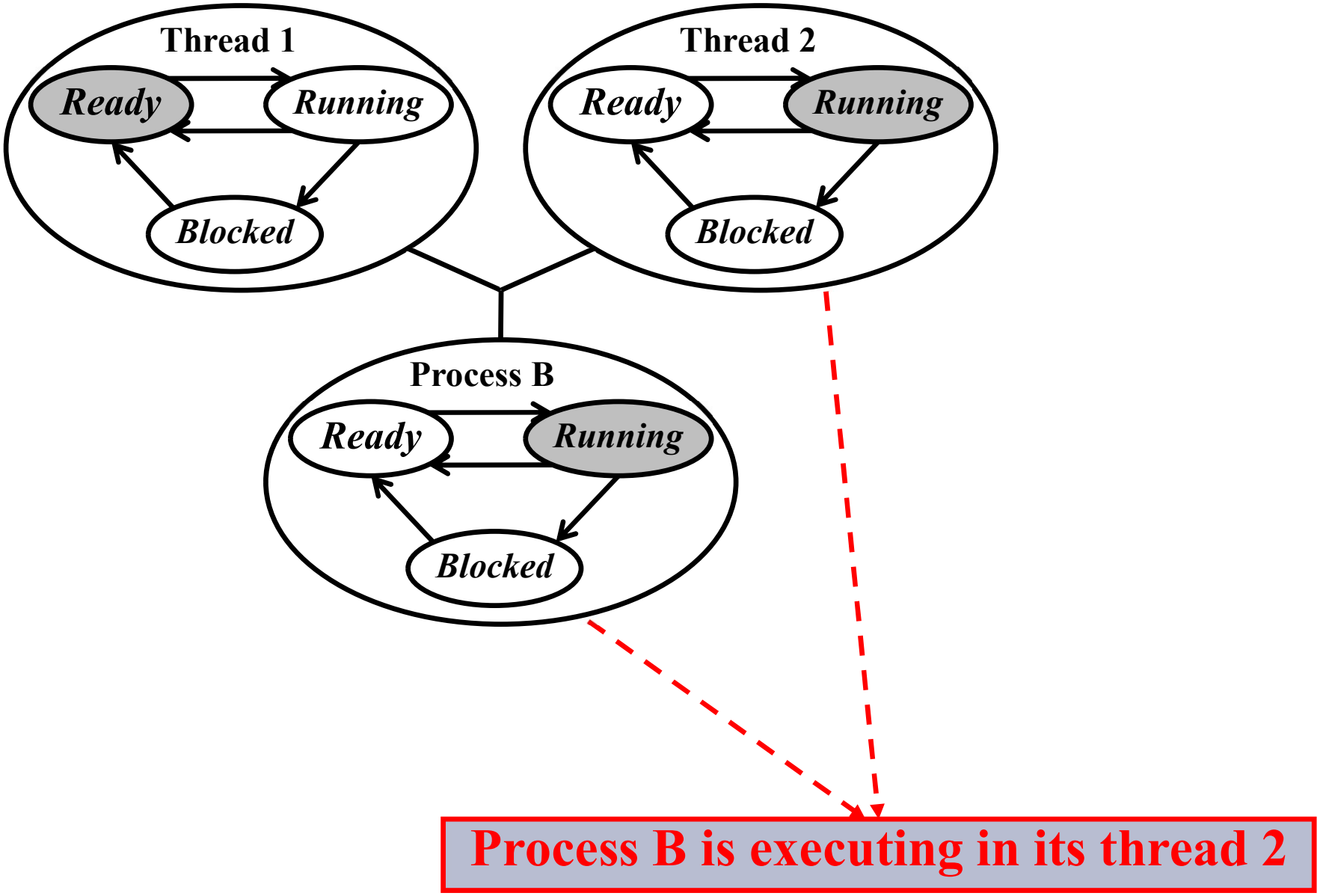
Each process has its own private thread table to keep track of the threads in that process:

It keeps track of the per-thread properties: each thread's program counter, stack pointer, registers, state, ….

The Relationships between User-Level Thread States and Process States

- **An example: there are two user-level threads in process B.**
  - **Initially, process B is executing in its thread 2.**

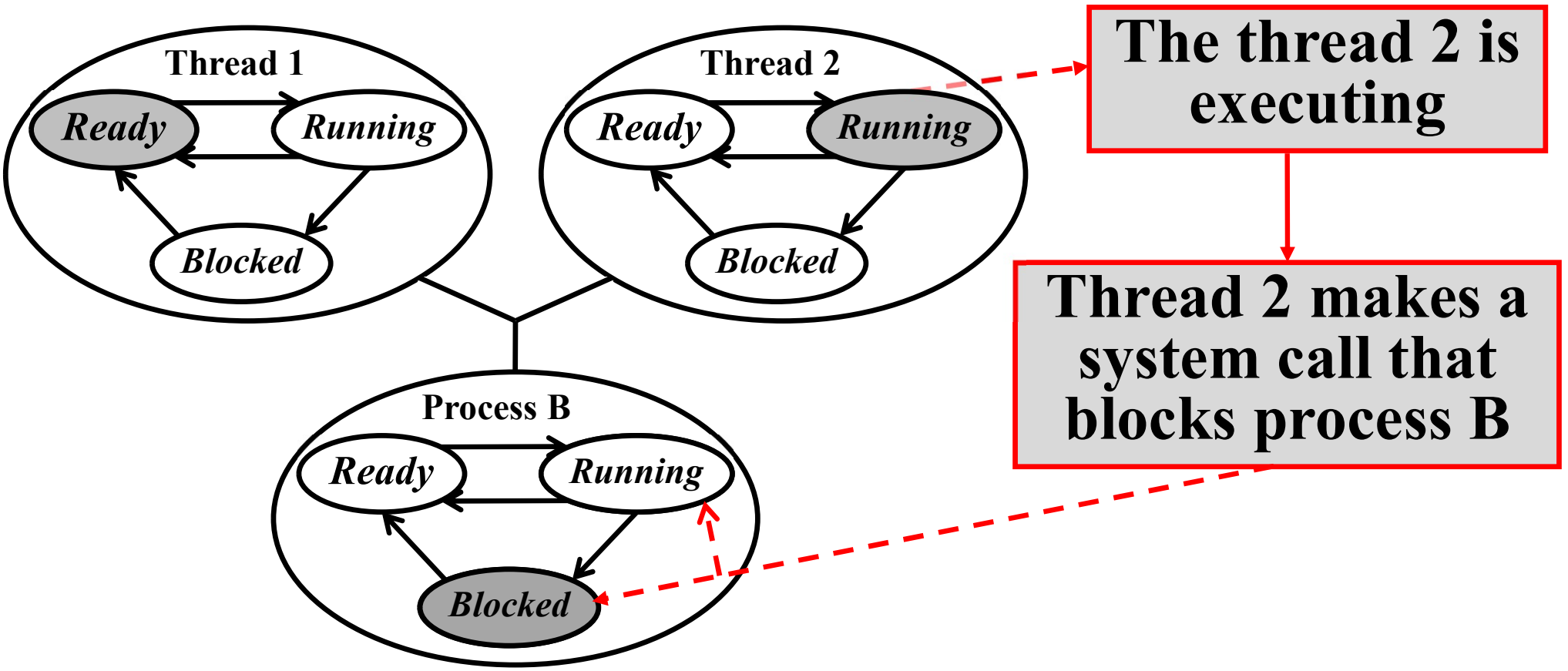


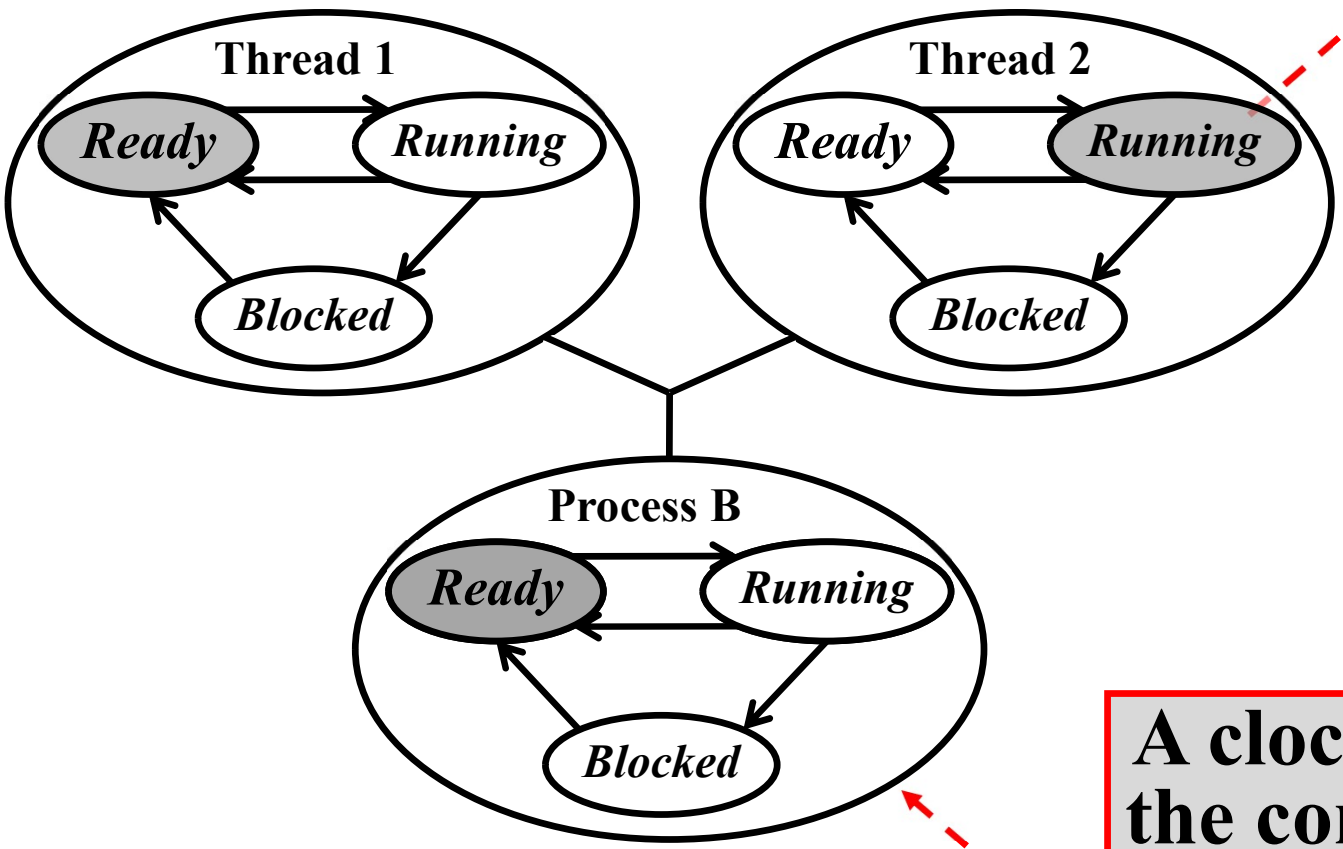

Thread 1
Ready
Running
Blocked
Thread 2
Ready
Running
Blocked
Process B
Ready
Running
Blocked
Process B is executing in its thread 2

Problems

- What happens for the following situations respectively?
  - Thread 2 makes a blocking system call
  - A clock interrupt passes control to the kernel and the kernel determines the currently-running process has exhausted its time slice.
  - Thread 2 has reached a point where it needs some action performed by thread 1 of process B.

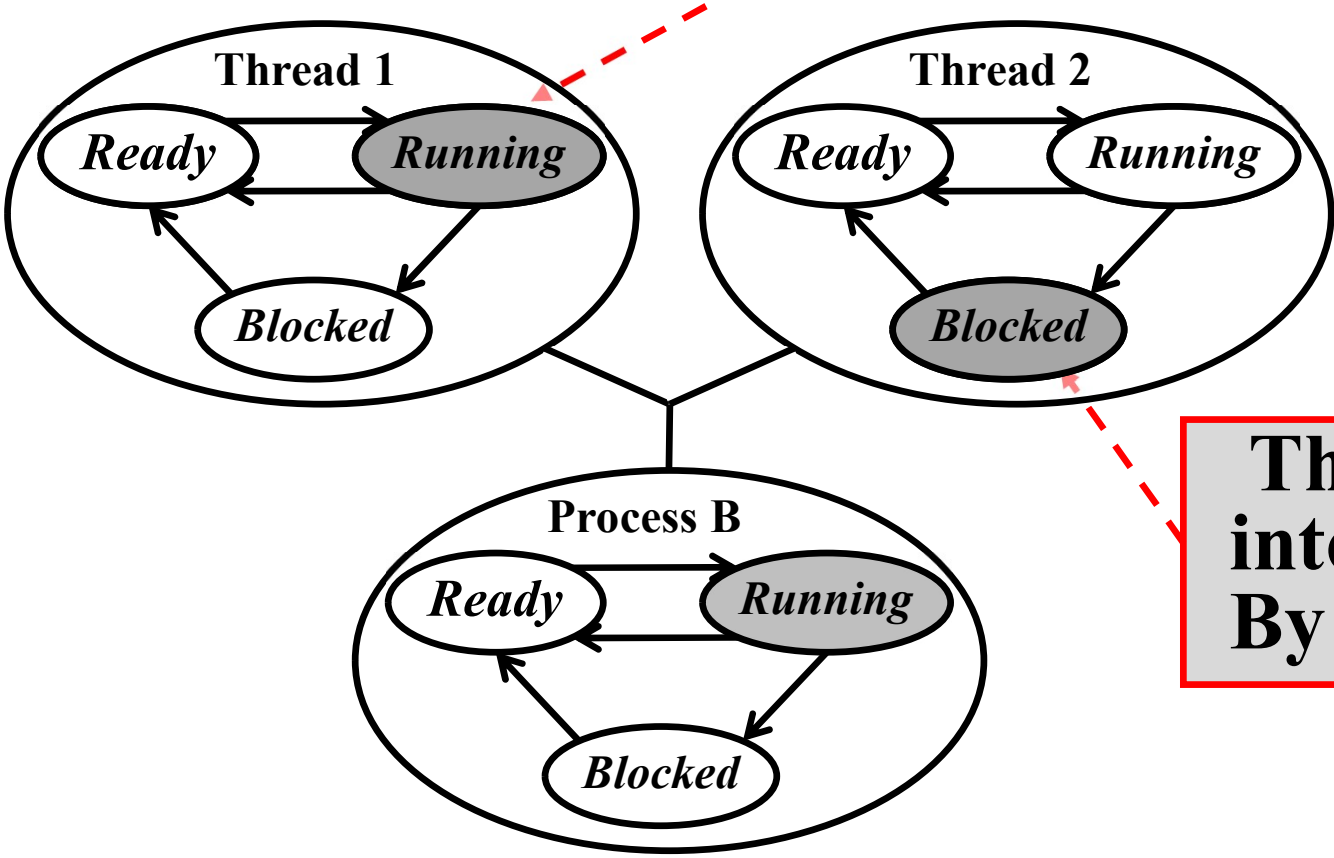



Thread 1
Ready
Running
Blocked
Thread 2
Ready
Running
Blocked
Process B
Ready
Running
Blocked
The thread 2 is executing
Thread 2 makes a system call that blocks process B



The thread 2 is executing

A clock interrupt passes the control to the kernel, and the kernel determines that the process B has exhausted its time slice



Thread 1 goes from Ready to Running

The thread 2 gets into Blocked state, By threads library

The thread 2 is waiting for some action performed by thread 1.

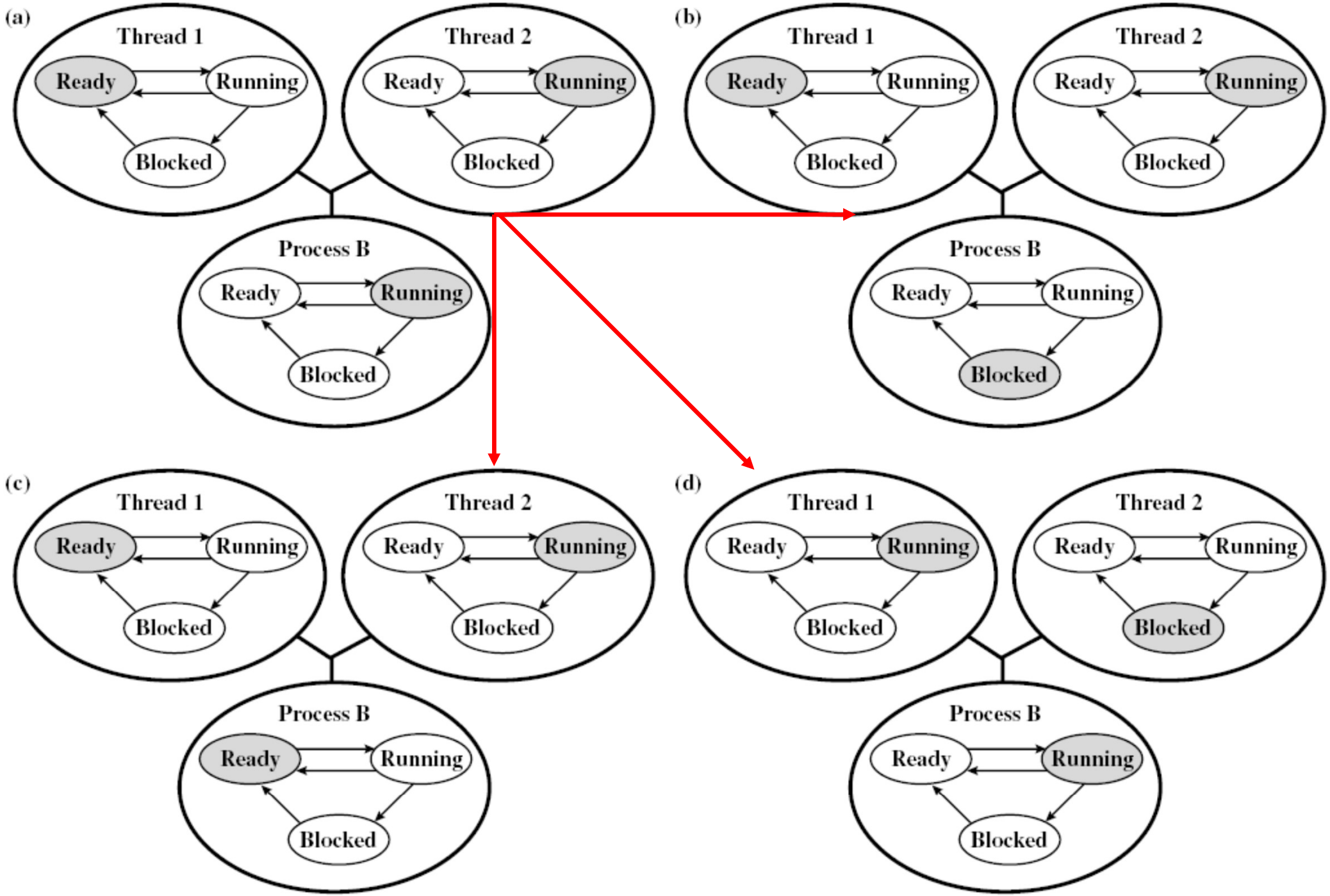


Figure 4.7 Examples of the Relationships Between User-Level Thread States and Process States

Advantages of ULTs

- Thread switching does not require switching to the kernel mode
  - Because all of the thread management data structures are within the user address space.
- Scheduling can be application-specific
- ULTs can run on any OS.



Disadvantages of ULTs

- **When ULT executes a system call and gets blocked, all of the threads in the process are blocked**
  - **Many system calls are blocking, in a typical OS.**
- **The problem of page faults**
  - **when a thread causes a page fault and other threads in the process might be runnable**
- **A multithreaded application can not take advantage of multiprocessing**
  - **Only a single thread within a process can execute at a time**

The management of multiple processes within a multiprocessor



Blocking System Calls

- Make system calls to be nonblocking
  - For example: a read on the keyboard would just return a 0 bytes if no characters were already buffered.
- Jacketing or Wrapper: to tell in advance if a system call will block. (… next slide)



Jacketing Technique

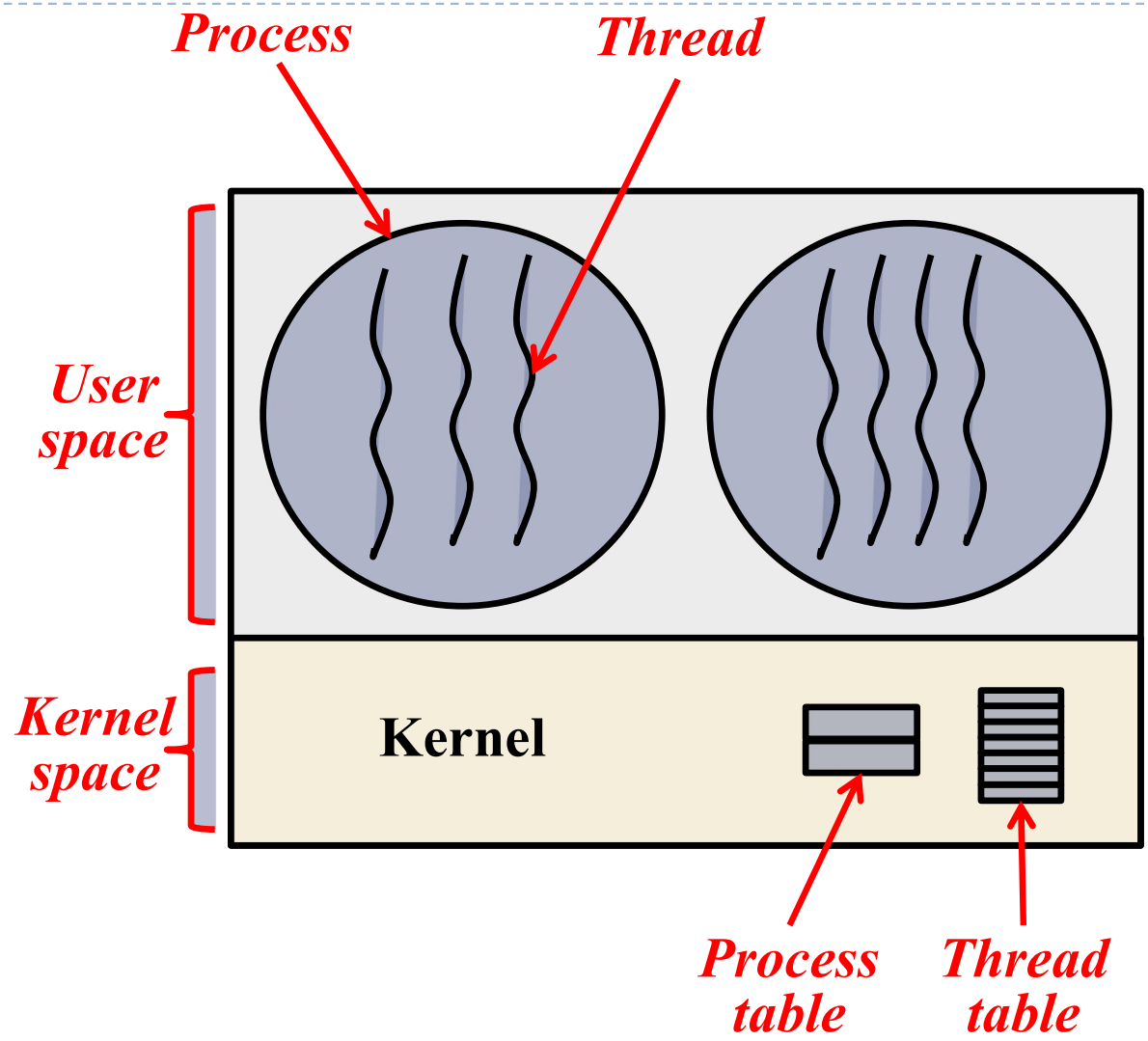
- Jacketing is one way to overcome the problem of blocking threads
- The purpose of jacketing is to convert a blocking system call into a nonblocking system call
  - A thread call an application-level I/O jacket routine, instead of calling system I/O routine
  - The jacket routine checks to determine if the I/O device is busy
    - If yes, this thread enters Ready state, and passes control to another thread
    - This thread checks the I/O device again when it is given control again.



Kernel-Level Threads

- **Pure KLT: All of the work of thread management is done by the kernel**
  - **W2K, Linux, and OS/2 are examples of this approach**
- **Kernel maintains context information for the process and the threads**
  - **There is no thread table in each process**
- **Scheduling is done on a thread basis**




Process
Thread
User space
Kernel space
Kernel
Process table
Thread table

Advantages of KLTs

- Simultaneously schedule multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process
- The kernel routines themselves can be multithreaded



Disadvantages of KLTs

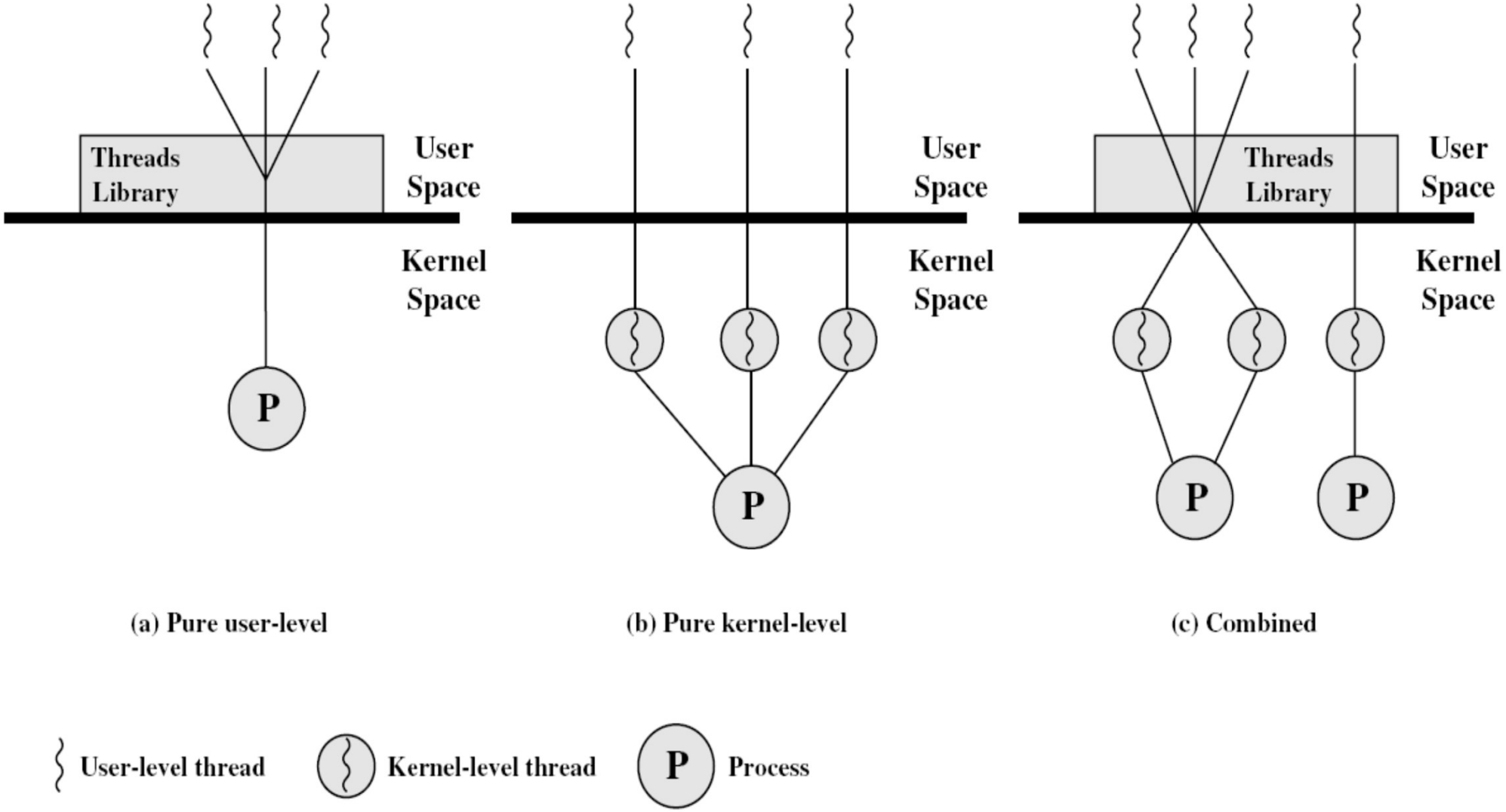
- All calls that might block a thread are implemented as system calls
  - Thus have considerably greater cost than a call to a run-time system procedure
- Transfer of control from one thread to another within the same process requires a mode switch to the kernel.



Relationships between User Threads and Kernel Threads

- Many-to-One Model:
  - Maps many user-level threads to one kernel threads
- One-to-One Model:
  - Maps each user thread to a kernel thread.
- Many-to-Many Model:
  - Multiplexes many user-level threads to a smaller or equal number of kernel threads




Threads
Library
User
Space
Kernel
Space
P
User
Space
Kernel
Space
P
Threads
Library
User
Space
Kernel
Space
P
P
(a) Pure user-level
(b) Pure kernel-level
(c) Combined
User-level thread
Kernel-level thread
P
Process

Organization

- An Introduction
- Multithreading Models
- **Thread Libraries**
- Threading Issues



Thread Library

- **Thread library**: provides programmers with an API for creating and managing threads
- **Two primary kinds of implementation:**
  - Provides a library entirely in user space with no kernel support
  - Implements a kernel-level library supported directly by the OS



Main Thread Libraries

- **Three main thread libraries are in use:**
  - **POSIX Pthreads**
    - **The threads extension of the POSIX standard: may be either a user- or kernel-level library**
  - **Win32**
    - **A kernel-level library available on Windows systems**
  - **Java**
    - **The Java thread API is typically implemented using a thread library available on the host system**



```
int sum;
void *runner(void *param)

int main(int argc, char *argv[])
{
   pthread_t tid;  /*the thread identifier
   pthread_attr_t attr; /*set of thread attributes
   if ( argc!=2 || atoi(argv[1])<0 ){
        //display error message and return;
   }
   pthread_attr_init(&attr); //get the default attributes
   pthread_create(&tid, &attr, runner, argv[1]); //create the thread
   pthread_join(tid, NULL); //wait for the thread to exit
   printf("sum = %d\n", sum);
}

void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;
   for (i=0; i<=upper; i++) sum+=i;
   pthread_exit(0);
}
```

Pthreads: An Example



```
DWORD Sum; //this data is shared by the thread(s)
DWORD WINAPI Summation(LPVOID Param){
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i=0; i<=upper; i++) Sum+=i;
    return 0;
}
int main(int argc, char *argv[]){
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;
    if ( argc!=2 || atoi(argv[1])<0 ){
        //display error message and return;
    }
    Param = atoi(argv[1]);
    ThreadHandle = CreateThread(        创建线程
        NULL, //default security attributes
        0, //default stack size
        Summation, //thread function
        &Param, //parameter to thread function;
        0, //default creation flags
        &ThreadId);//return the thread identifier
    if (ThreadHandle!=NULL){
        //wait for the thread to finish
        WaitForSingleObject(ThreadHandle, INFINITE);
        CloseHandle(ThreadHandle);  //close the thread handle
        printf("sum = %d\n", Sum);
    }
}
```

让父线程阻塞，直到累加线程退出

Win32 Threads: An Example

Organization

- An Introduction
- Multithreading Models
- Thread Libraries
- **Threading Issues**

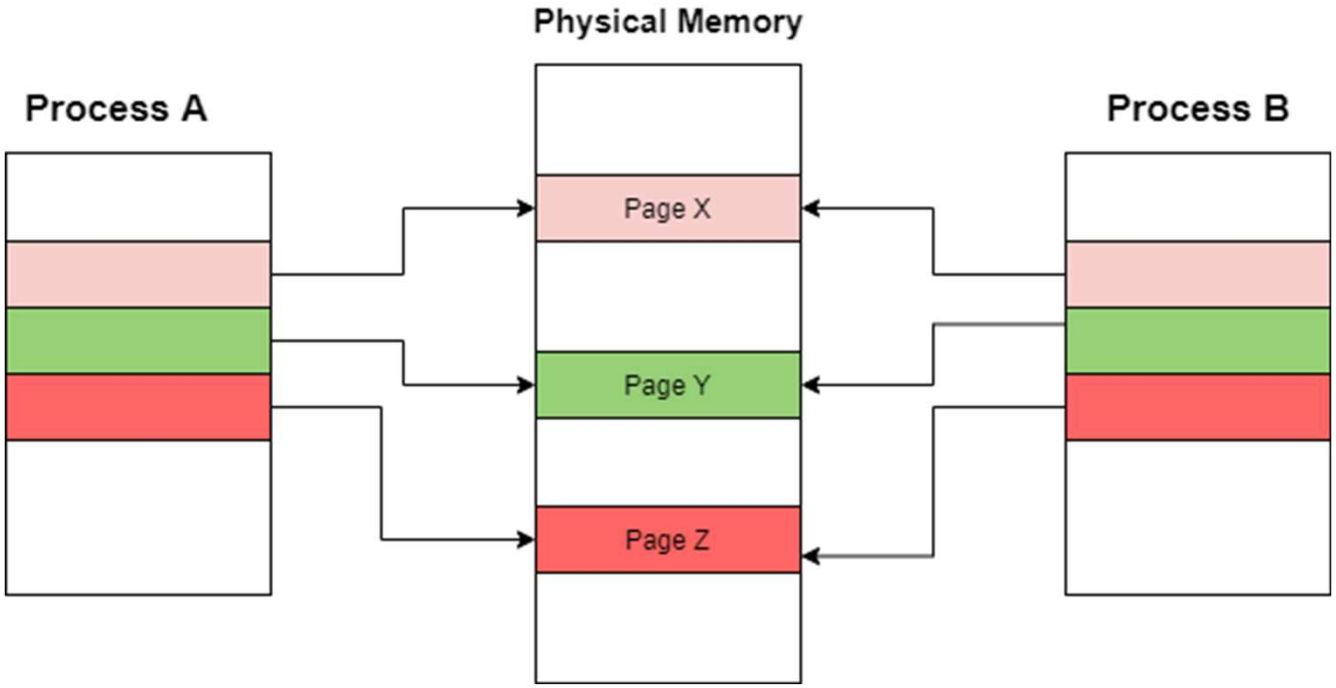


Threading Issues

- **The `fork()` system call**
- **Thread Cancellation**
- **Signal Handling**
- **Thread Pools**
- **Scheduler Activation**

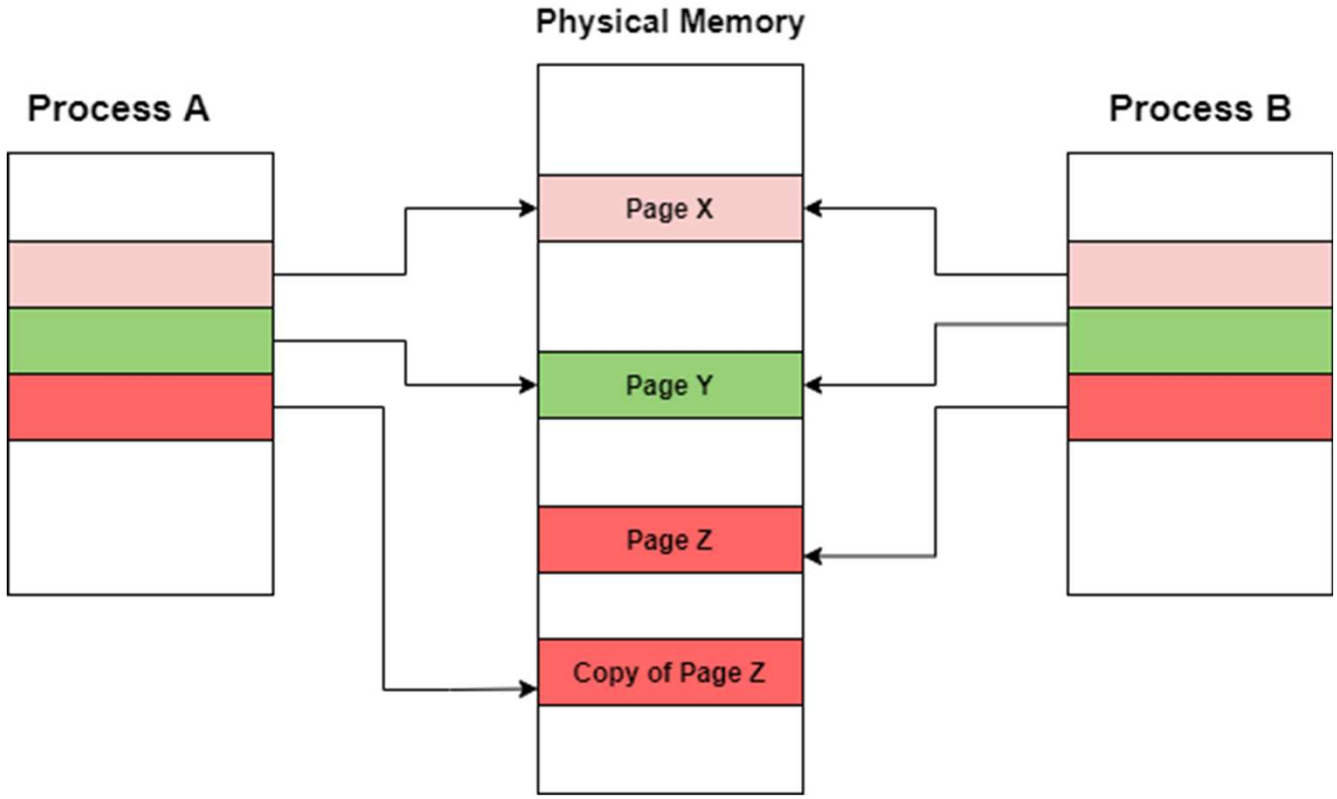


Copy-On-Write





Copy-On-Write





The `fork()` System Call

- Two choices:
  - Does the new process duplicate all threads?
  - Or is the new process duplicate only the thread that invoked `fork()` system call?
- The decision depends on the application
  - Whether the `exec()` is called immediately after forking



Thread Cancellation
What and Why?

- What is Thread Cancellation?
  - Terminating a thread before it has completed.

- Scenario Examples:
  - Example 1: Multiple threads are concurrently searching through a database, and one thread returns the result, the remaning threads might be canceled.

  - Example 2: A user presses a button on a web browser that stops a web page from loading any further.



Thread Cancellation
Two general approaches

- Thread to be canceled is **target thread**

- Difficulties:
  - When **resources have been allocated** to a cancelled thread
  - When a thread is canceled **while in the midst of updating data it is sharing with other threads**

- Two general approaches:
  - **Asynchronous cancellation** - One thread immediately terminates the target thread
  - **Deferred cancellation** - The target thread periodically checks whether it should terminate, allowing it an opportunity to terminate itself in an orderly fashion



Thread Cancellation Pthreads

```
pthread t tid;
/* create the thread */
pthread create(&tid, 0, worker, NULL);
…
/* cancel the thread */
pthread cancel(tid);
```

Invoking pthread cancel() indicates only a request to cancel the target thread.

Actual cancellation depends on how the target thread is set up to handle the request.

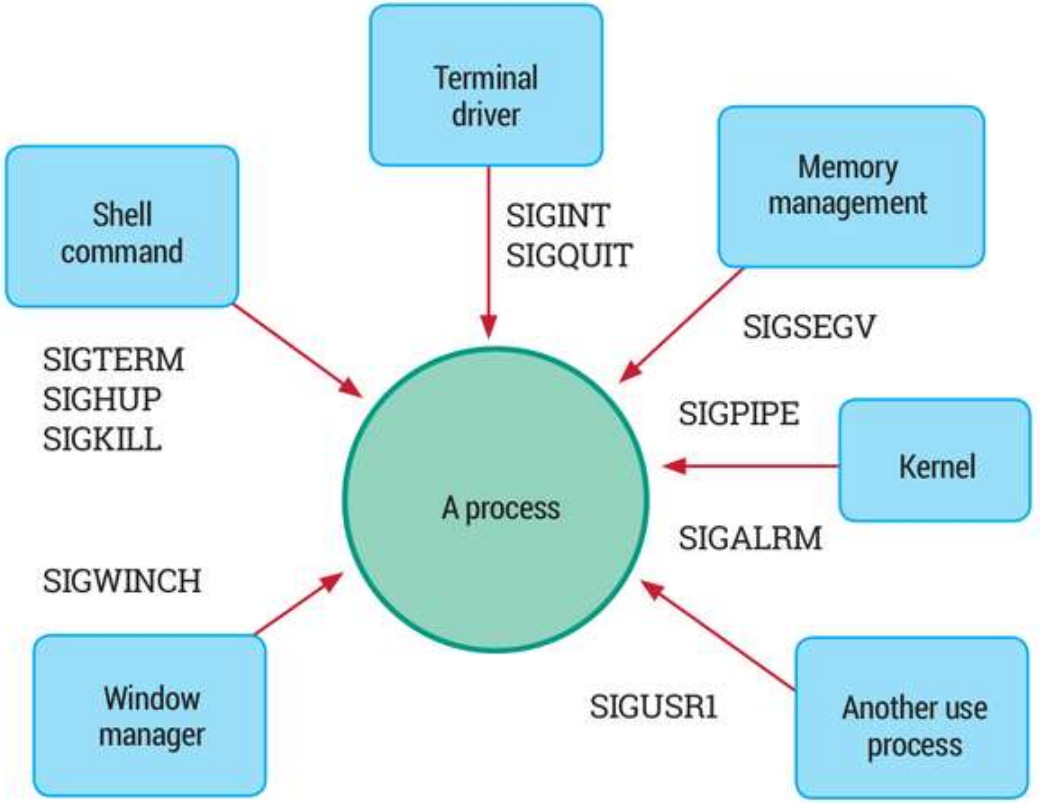
Mode	State	Type
Off	Disabled	-
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

```
while (1){
  /* do some work for a while */
  /* ... */
  /* check if there is a cancellation request */
  pthread testcancel();
}
```



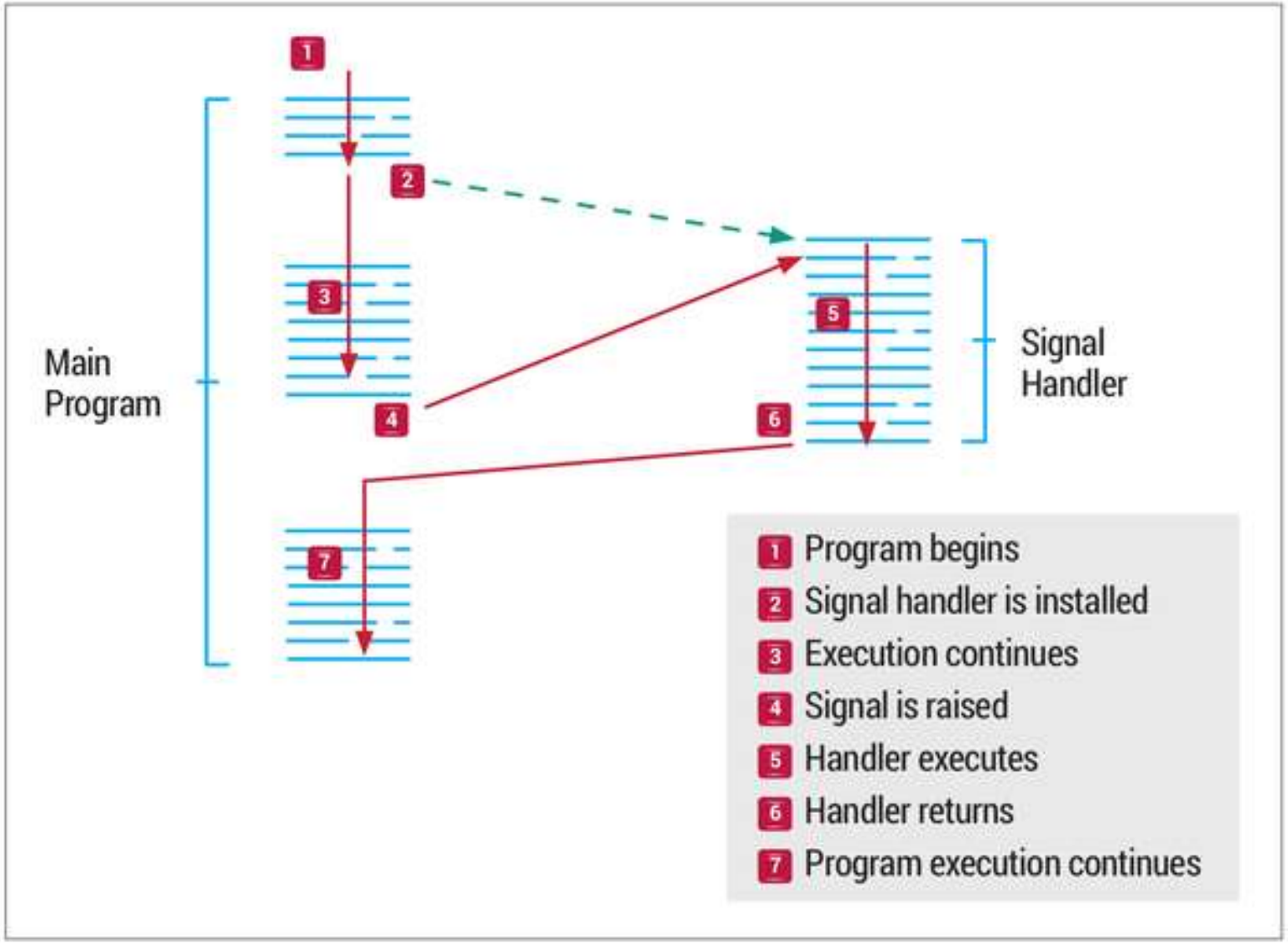
Signal

Signal (信号) is basically a one-way notification (单向通知)



SIGINT	2	Interrupt a process (used by **Ctrl-C**)
SIGHUP	1	Hang up or shut down and restart process
SIGKILL	9	Kill the process (cannot be ignored or caught elsewhere)
SIGTERM	15	Terminate signal, (can be ignored or caught)
SIGTSTP	20	Stop the terminal (used by **Ctrl-z**)
SIGSTOP	19	Stop execution (cannot be caught or ignored)

Signal Handling



```c
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void handle_sigint(int sig, siginfo_t *si, void *uc) {
    printf("Caught signal %d\n", sig);
}

int main() {
    struct sigaction sa;
    sa.sa_flags = SA_SIGINFO;
    sa.sa_sigaction = handle_sigint;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    while (1) {
        printf("Hello, World!\n");
        sleep(1);
    }
    return 0;
}
```

sigaction 函数使用一个 sigaction 结构体来指定信号的处理方式，这个结构体包含了信号处理函数、信号掩码和其它选项

`sigaction()` sets up the signal handler function `handle_sigint` for SIGINT

Signal Handling

- **A signal** is used in UNIX to **notify a process that a particular event has occurred**.
  - Signals interrupt whatever the process is doing at this minute, and force it to handle them immediately.
  - Each signal has an integer number that represents it (1, 2, …)
  - Each signal may have a signal handler, which is a function that gets called when the process receives that signal
- Two kinds of signals
  - **Sychronous Signal (同步信号)**: generated by **exceptions** or errors, which are caused by invalid operations in a process.
    - Examples: Divide-by-Zero, Illegal Memory Access
  - **Asynchronous Signal (异步信号)**: generated by an **event external** to a running process.
    - Example: Timer Expiration,



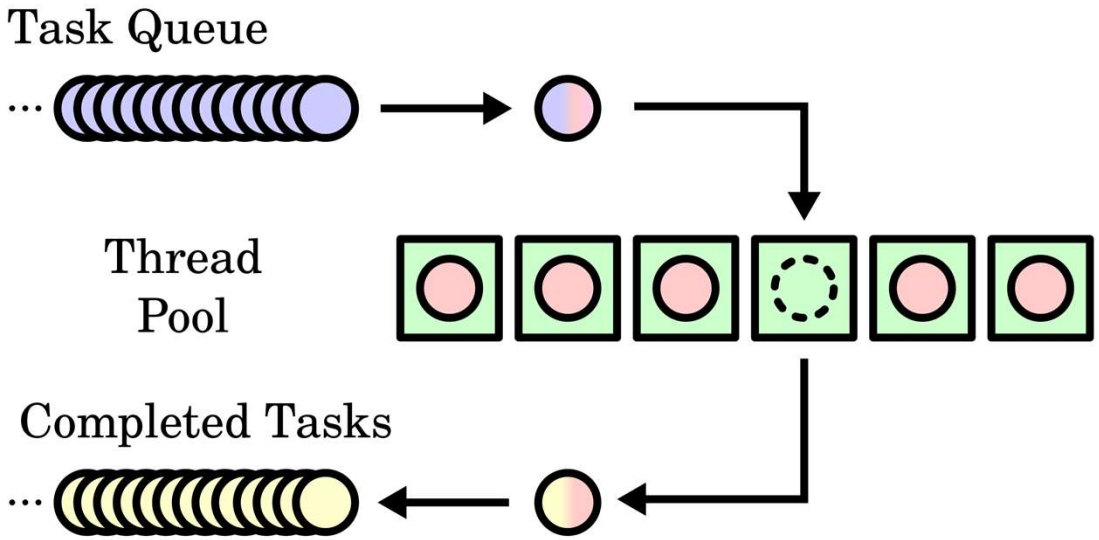
Signal Handling

- To extend signal handling to multi-threaded environments, since a process may have several threads, where should a signal be delivered?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process.

- **It depends on the type of signal generated**!



Thread Pools



- Basic idea of thread pool:
  - To **create a number of threads at process startup** and place them into a pool, where the threads sit and wait for work
  - **When a server receives a request**, it **awakens** a thread from the pool and passes it the request to service
  - Once the thread **complete its service**, it returns to the pool and **awaits more work**



Thread Pools

- Benefits:
  - Servicing a request with an existing thread is **usually faster** than waiting to create a thread
  - A thread pool **limits the number of threads** that exist at any one point. This is important for systems that cannot support a large number of concurrent threads.



Scheduler Activation (1)
调度器激活机制

- **Many-to-Many Model**: requires communication between kernel and thread library.
  - Place an intermediate data structure (called lightweight process or LWP) between the user and kernel threads.
  - To the user-thread library, the LWP appears to be a virtual processor.
    The application can schedule a user thread to run on a virtual processor.
  - Each LWP is attached to a kernel thread.
    The OS schedules kernel threads to run on physical processors.

- An application may require any number of LWPs to run efficiently
  - On a uniprocessor system, one LWP is enough for a CPU-bound application.
  - However, I/O-intensive application may require multiple LWPs to execute.



Scheduler Activation (2)
Upcall (上行调用)

- Scheduler activation: for communication between the user-thread library and the kernel
  - The kernel provides an application with a set of LWPs (or virtual processors)
  - The application can schedule user threads onto an available virtual processor.
  - **(重要) The kernel must inform an application about certain events**, which is called an **updcall**.
    - The thread library handles upcalls with an upcall handler



Scheduler Activation (2)

- The steps:
  - One event that triggers **an upcall** occurs **when an application thread is about to block**. Then the kernel makes an upcall to the application
  - The kernel then allocate a new virtual processor to the application. The application runs an upcall handler on it.
  - The upcall handler **saves the state of the blocking thread** and **relinquishes the virtual processor of the blocking thread**
  - The upcall handler then schedules another thread that is eligible to run on the virtual processor.
  - **When the event that the blocking thread was waiting for occurs**, the kernel makes **another upcall** to the thread library, informing it that the previously **blocked thread is now eligible to run**.

